```
        "akka.projection.jdbc.blocking-jdbc-dispatcher"))    ❶

  def getBetByMarket(
      in: MarketIdsBet): Future[SumStakes] = ...
  ...
}
```

❶ Separate executor to provide the future computation for getBetByMarket

Once the .proto that uses the `BetRepository` `SELECT` query is implemented, you just need to expose it. Choose a port and a host, and bind it to an Akka HTTP server.

Listing 15.4 Bet projection server

```
object BetProjectionServer {

  def init(repository: BetRepository)(
      implicit ec: ExecutionContext,                          ❶
      system: ActorSystem[_]): Unit = {
    val service: HttpRequest => Future[HttpResponse] =
      BetProjectionServiceHandler.withServerReflection(
        new BetProjectionServiceImpl(system, repository))     ❷

    val port =
      system.settings.config.getInt("services.bet-projection.port")
    val host = system.settings.config.getString("services.host")

    Http().newServerAt(host, port).bind(service)              ❸
}}
```

❶ Execution context for the bind that returns a future

❷ Uses the .proto service implementation

❸ Starts the server and binds the service

The `BetProjectionServiceHandler` wraps the implementation of the service into a function of `HttpRequest => Future[HttpResponse]`, as specified in the signature of the service value. To start the server, as mentioned earlier, make sure you have a `ScalikeJdbcSession` in scope with